SONG LYRICS GENERATOR

MICHAŁ EJANKOWSKI & DANIIL MARTINOVICH

MINI-PROJECT PRESENTATION

PROBLEM STATEMENT

Contemporary songwriters often struggle with **idea generation**: transforming an abstract theme into the first lines of lyrics. Automating this creative “warm-up” requires **language technology** that can respect topical constraints, poetic metre and rhyme schemes.

GOAL

Develop a prototype tool that generates English song lyrics conditioned on a topic or affect keyword with accurate rhyming. Output should be coherent, stylistically consistent and end each line with a plausible rhyme.

RELATION TO NLP

The task combines neural text generation (large-language-model fine-tuning) with phonetic processing (detecting rhymes via CMU-style pronunciations).

PLANNED SOLUTION

MODEL

Fine-tune **GPT-2 Small** (124 M) on a curated subset of song lyrics using LoRA to keep compute under 8 GB GPU RAM.

RHYME ENGINE

After each generated line, inspects the final token and gives alternatives from *syllapy* and *G2p coverter*

INTERFERENCE LOOP

Generate iteratively line-by-line to allow rhyme control with simple Python interface

DATASET

Mendeley Data

You are viewing a previous version of this dataset

Music Dataset: Lyrics and Metadata from 1950 to 2019

Published: 24 August 2020 | Version 2 | DOI: 10.17632/3t9vbwxgr5.2

Contributors: Luan Moura, Emanuel Fontelles, Vinicius Sampaio, Mardônio França

Music Dataset: Lyrics and Metadata 1950–2019 (~95 k songs). We filter English entries ≥ 5 lines and optionally a single genre (e.g., Blues) to minimise domain shift.

Music Dataset: Lyrics and Metadata from 1950 to 2019

This dataset provides a list of lyrics from 1950 to 2019 describing music metadata as sadness, danceability, loudness, acousticness, etc. We also provide some informations as lyrics which can be used to natural language processing. The audi…

Mendeley Data / Aug 24, 2020

WORKPLAN

1

DATA & MODEL SETUP

- Download GPT-2 Small + lyrics dataset.
- Clean corpus: English only, ≥ 5 lines, deduplicate.

2

FINE-TUNING

- Apply LoRA tuning (≈ 20 epochs) on 8 GB GPU; save checkpoint.

3

RHYME GENERATION PROTOTYPE

- Implement iterative line-by-line decoding.
- Post-filter last word with g2p converter

4

EVALUATION

- Automatic: A/B testing comparing the model to baseline (GPT-2 without rhyme filter)
- Human: 2 volunteers subjectively rate coherence, rhyme quality and creativity

RELATED WORK

Lyric jam - generates lyrics real-time using AI by listening the incoming audio.

Lyrics Generation Applying Metaphor Generation - generates lyrics with metaphors.

Other developments: multilingual support, full song creation, support for specific genre

TABLE OF CONTENTS

Table of contents

- Preparation
  - Setup
  - LoRA
  - GPT2 small
  - Dataset conversion
    - Modification
    - Get values necessary for data processing
    - Convert data into a training prompt for GPT
    - Create dataset for GPT
    - Load the dataset
- Training
  - Prepare training
    - Google Drive mount
    - Check if GPU is used
    - Define training
  - Train
- Usage

› Preparation

↳ 17 cells hidden

› Training

[ ] ↳ 19 cells hidden

› Usage

[ ] ↳ 21 cells hidden

› Final result

[ ] ↳ 1 cell hidden

› Evaluation

[ ] ↳ 14 cells hidden

PREPARATION

NECESSARY DOWNLOADS

```
!wget -nc --header="User-Agent: Mozilla/5.0" https://data.mendeley.com/public-files/datasets/3t9vbwxgr5/files/d2c58546-d8be-4d57-af14-b61337c927b2/file_downloaded -O "data.csv"
```

```
--2025-06-11 11:32:57--  https://data.mendeley.com/public-files/datasets/3t9vbwxgr5/files/d2c58546-d8be-4d57-af14-b61337c927b2/file_downloaded
Resolving data.mendeley.com (data.mendeley.com)... 162.159.130.86, 162.159.133.86
Connecting to data.mendeley.com (data.mendeley.com)|162.159.130.86|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com/036e1862-4d9b-4ea8-b762-7ae3282ba2c3 [following]
--2025-06-11 11:32:58--  https://prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com/036e1862-4d9b-4ea8-b762-7ae3282ba2c3
Resolving prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com (prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com)... 3.5.66.119, 52.92.17.90, 52.92.32.170, ...
Connecting to prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com (prod-dcd-datasets-public-files-eu-west-1.s3.eu-west-1.amazonaws.com)|3.5.66.119|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27655251 (26M) [text/csv]
Saving to: ‘data.csv’

data.csv            100%[===================>]  26.37M  12.0MB/s    in 2.2s

2025-06-11 11:33:01 (12.0 MB/s) - ‘data.csv’ saved [27655251/27655251]
```

```
[ ] !pip install nltk pronouncing
```

```
[ ] !pip install --upgrade transformers
```

```
[ ] !pip install datasets
```

```
[ ] !pip install pyphen
```

```
[ ] !pip install syllapy
```

```
[ ] !pip install peft
```

LoRA

```
[ ] from peft import get_peft_model, LoraConfig, TaskType

    def applyLoRA(model):
      # LoRA configuration
      lora_config = LoraConfig(
          r=8,                # rank of LoRA matrices
          lora_alpha=32,      # scaling factor
          target_modules=["c_attn"],  # GPT2 attention layers to adapt
          lora_dropout=0.1,
          bias="none",
          task_type=TaskType.CAUSAL_LM,
      )
      # Apply LoRA to model
      return get_peft_model(model, lora_config)
```

LORA SETUP

```
[ ] from transformers import GPT2Tokenizer, GPT2LMHeadModel
    from transformers import DataCollatorForLanguageModeling

    # Load GPT2 small and tokenizer
    model_name = "gpt2"
    tokenizer = GPT2Tokenizer.from_pretrained(model_name)
    model = GPT2LMHeadModel.from_pretrained(model_name)

    # Add pad token to GPT2, to deal with text of varying length
    tokenizer.pad_token = tokenizer.eos_token
    model.config.pad_token_id = model.config.eos_token_id

    # Apply LoRA to the model
    model = applyLoRA(model)

    # Prepare data collator
    data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False) # GPT won't use mask tokens
```

GPT2 SMALL SETUP

GENRE REDUCTION TO: BLUES

```python
# Load CSV
raw_df = pd.read_csv("data.csv")
print("All genres:", raw_df['genre'].unique())

# Leave only one genre
global GENRE
GENRE = 'blues'
raw_df = raw_df[raw_df['genre'] == GENRE] # reduce the dataset size
raw_df = raw_df[raw_df['obscene'] < 0.1] # reduce the amount of vulgar words
print(f"The most rude lyric in the resulting dataset: ", raw_df.loc[raw_df['obscene'].idxmax()]["lyrics"])
print(f"Amount of rows for {GENRE}:", raw_df.shape[0])

# Pick all keywords first
df = raw_df.iloc[:,7:-2]
print("\nOld keywords:\n")
print(df.max())

# Clear the data
df = df.drop(columns=['loudness', 'acousticness', 'instrumentalness', 'communication', 'music']) # not descriptive for lyrics
df = df.drop(columns=['shake the audience', 'like/girls', 'family/gospel', 'family/spiritual', 'light/visual perceptions', 'movement/places', 'dating']) # will not provide enough examples (max value too low)
df = df.drop(columns=['obscene']) # obscene lyrics are filtered out

# Convert keywords to their adjective form for prompting
df = df.rename(columns={'valence': 'happy'})  # the higher values for valence mean uplifting
df = df.rename(columns={'energy': 'energetic'})
df = df.rename(columns={'danceability': 'danceable'})
df = df.rename(columns={'violence': 'violent'})
df = df.rename(columns={'sadness': 'sad'})
df = df.rename(columns={'feelings': 'emotional'})
df = df.rename(columns={'world/life': 'timeless'})
df = df.rename(columns={'night/time': 'temporal'})  # means related to time
print("\nUpdated keywords:\n")
print(df.max())
```

DATA TOKENIZATION

Load the dataset

```
[ ] from datasets import load_dataset

    # Tokenize dataset
    def tokenize_function(examples):
        return tokenizer(examples["text"], truncation=True, max_length=512)

    def load_tokenized_dataset():
      return dataset.map(tokenize_function, batched=True, remove_columns=["text"])

    # Load the dataset
    tokenized_dataset = load_tokenized_dataset()
```

```
Map: 100% ■■■
```

TRAINING

```python
from transformers import Trainer, TrainingArguments

# Training args
training_args = TrainingArguments(
    weight_decay=0.01,                # Prevent overfitting
    output_dir=SAVE_PATH,         # Save directory
    num_train_epochs=6,               # Reasonable for small dataset
    per_device_train_batch_size=4,    # Lower batch size = fits in memory
    gradient_accumulation_steps=4,    # Simulates larger batch (4x4=16)
    logging_steps=1,                 # Print loss every 50 steps
    save_steps=1,                   # Save checkpoint every 200 steps
    save_total_limit=1,               # Keep only 1 checkpoint
    learning_rate=5e-5,               # Good starting point
    fp16=False,                       # Set True if using supported GPU
    report_to="none",                 # Avoid W&B prompts
    overwrite_output_dir=True,        # Replace previous training run
)
training_args.evaluation_strategy = "steps" # set "steps" or "epoch" if you have validation

# Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    data_collator=data_collator,
)
```

```python
# Training
trainer.train(resume_from_checkpoint=chk_pnt)
```

[1362/1362 10:19, Epoch 6/6]

Step	Training Loss
706	4.044900
707	4.014400
708	3.970300
709	4.698500
710	4.373100
711	4.298400
712	4.463600
713	3.597800
714	4.007900
715	3.914000
716	4.352200
717	3.793500
718	4.721000
719	4.338000
720	4.078000
721	3.945000
722	4.036800
723	4.390300
724	4.117600

POST-PROCESSING AND FINE-TUNING

PROMPT-ENGINEERING

```python
import random

def make_inference_prompt(topic_index, keyword_indexes):
  input = {}

  # Set topic
  input["topic"] = TOPICS[topic_index]

  # Set keywords
  for keyword in KEYWORDS:
    input[keyword] = 0.00
  for keyword_index in keyword_indexes:
    input[KEYWORDS[keyword_index]] = 1

  # Set lyrics
  input["lyrics"] = "."

  # Create prompt
  return make_prompt(input)[:-2]

# Example
topic = 4
keywords = [5,8]
print("Topic:",topic,"-",TOPICS[topic])
print("Keywords:",[(lambda x: str(x)+" - "+KEYWORDS[x])(x) for x in keywords])
print("\nInference prompt:")
print(make_inference_prompt(topic,keywords))
```

```
Topic: 4 - romantic
Keywords: ['5 - emotional', '8 - energetic']

Inference prompt:
Generate a line for a blues song about a romantic topic with emotional and energetic feel.
```

LORA AND GPT2 MERGE

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel, PeftConfig

def load_checkpoint(num, train=False):
  # Load tokenizer (same as original model)
  tokenizer = AutoTokenizer.from_pretrained("gpt2")

  # Load LoRA config from checkpoint
  if train:
    peft_model_path = SAVE_PATH + "/checkpoint-" + str(num) # for mounted GDrive
  else:
    peft_model_path = "gpt2-lora/checkpoint-" + str(num)

  config = PeftConfig.from_pretrained(peft_model_path)

  # Load base model with LoRA adapter
  base_model = AutoModelForCausalLM.from_pretrained(config.base_model_name_or_path)
  model = PeftModel.from_pretrained(base_model, peft_model_path)
  return tokenizer, model
```

```python
# Inference
def infer_lyrics(prompt):
  inputs = tokenizer(prompt, return_tensors="pt")
  outputs = model.generate(
      **inputs,
      max_length=300,
      temperature=0.7,           # adds some creativity, but not too random
      top_k=50,                  # restricts to top-k likely tokens
      top_p=0.9,                 # nucleus sampling: focus on top 90% prob mass
      do_sample=True,            # enables sampling
      num_return_sequences=1,    # only 1 line
      repetition_penalty=1.1,    # try to avoid endless repetition
  )
  return tokenizer.decode(outputs[0], skip_special_tokens=True)

# Example
topic = 4
keywords = [5]
print("Topic:",topic,"-",TOPICS[topic])
print("Keywords:",[(lambda x: str(x)+" - "+KEYWORDS[x])(x) for x in keywords])

example_prompt = make_inference_prompt(topic, keywords)
print("\nExample prompt:")
print(example_prompt)

print("Generated lyrics line:")
print(infer_lyrics(example_prompt))
```

```
Example prompt:
Generate a line for a blues song about a romantic topic with emotional feel.

Generated lyrics line:
Generate a line for a blues song about a romantic topic with emotional feel.
blue sky blue sun yellow baby green wind girl come fly wild leave go fall boy love look like child break away forget get hold r
```

FIRST LYRICS GENERATION

This generated lyrics that were not in any way rhyming, just randomized selection from the dataset

INITIAL RESULT

Warning: no stopwords in the dataset

POST-PROCESSING

RHYME GENERATION

```python
from g2p_en import G2p
import re
from collections import defaultdict

# Initialize G2p converter
g2p_converter = G2p()

# Phonetic rhyming logic
def get_phonetic_rhyming_part(phonemes):

    # Extract the 'rhyming part' from a list of ARPABET phonemes.
    vowel_pattern = re.compile(r'[AEIOU][0-2]?')

    for i in reversed(range(len(phonemes))):
        if vowel_pattern.match(phonemes[i]):
            return phonemes[i:]
    return []

# Check if two words approximately rhyme based on their G2P phonetic similarity.
def are_approx_rhymes(word1, word2, min_shared_phonemes=2):
    if word1.lower() == word2.lower(): # A word doesn't rhyme with itself
        return False

    phonemes1 = g2p_converter(word1.lower())
    phonemes2 = g2p_converter(word2.lower())

    if not phonemes1 or not phonemes2:
        return False

    rhyming_part1 = get_phonetic_rhyming_part(phonemes1)
    rhyming_part2 = get_phonetic_rhyming_part(phonemes2)

    if not rhyming_part1 or not rhyming_part2:
        return False
```

PHONETIC RHYME DETECTION

```python
# Example
sample_text = "The old gray cat sat by the mat. He wore a fine hat, a real treat."
example_search = "cat"
print("Sample text:", sample_text)
print("Search word:", example_search)
print(f"Rhymes for 'cat' (first 7 syllables ignored): {find_rhymes(example_search, sample_text, min_shared_phonemes=2)}")
```

```
Sample text: The old gray cat sat by the mat. He wore a fine hat, a real treat.
Search word: cat
Rhymes for 'cat' (first 7 syllables ignored): ['mat', 'hat']
```

EVALUATION AND RESULTS

```python
# Show choosable values
def lyrics_info():
  print("Topics:\n")
  for i, item in enumerate(TOPICS):
      print(f"{i}: {item}")
  print("\nKeywords:\n")
  for i, item in enumerate(KEYWORDS):
      print(f"{i}: {item}")
  print("\n")

# Primitive way to make rhyming lyrics
def generate_lyrics_chunk(topic_index, keyword_indexes):
  gpt_prompt = make_inference_prompt(topic, keywords)
  gpt_line = infer_lyrics(gpt_prompt)
  gpt_line = "\n".join(gpt_line.split("\n")[1:]) # remove the prompt part

  # Prepare the first line
  line = top_line(gpt_line) # take first 8 syllables
  last_word = line.strip().split()[-1] # take the last word to rhyme

  final_lines = [line.capitalize()+","]

  # Generate rhyming lines
  for i in range(0,3):
    # Generate a new rhyming line
    rhymes = []
    while len(rhymes) == 0:
      gpt_line = infer_lyrics(gpt_prompt)
      gpt_line = "\n".join(gpt_line.split("\n")[1:]) # remove the prompt part
      print(i,"line:",gpt_line)
      rhymes = find_rhymes(last_word, gpt_line, 1)
      print(i,"rhymes:",rhymes)
```

```
Topics:

0: music
1: violence
2: world/life
3: sadness
4: romantic
5: feelings
6: night/time

Keywords:

0: violent
1: timeless
2: temporal
3: romantic
4: sad
5: emotional
6: danceable
7: happy
8: energetic


Chosen topic: 4 - romantic
Chosen keywords: ['5 - emotional']
```

```
Generated lines:
Come away like cold stone come home,
Kiss hand remember tonight time,
Back inside night walk away dream,
Know what kind of heart hear right come,
```

SELECTION OF POSSIBLE TOPICS

SELECTION OF KEYWORDS

CHOSEN TOPIC AND KEYWORD

A/B COMPARING FINED-TUNED MODEL TO BASELINE GPT2

```
--- TEST CASE ---
Chosen topic: 2 - romantic
Chosen keywords: ['1 - timeless', '2 - temporal']

The prompt: Generate a line for a blues song about world/life with timeless and temporal feel.


GPT SMALL RAW:
The title of the project comes from an email sent to me by "Vladimir" Eustace, who is writing his first solo album as well as several songs that will be released in 2017 under IRL


FINE-TUNED GPT:
Love come love fall look see eye tell,
Like love stay hold leave mind wait till,
Love kiss goodbye hug heart walk leave soul,
Sleep know give say come out stay tell,


--- TEST CASE ---
Chosen topic: 3 - romantic
Chosen keywords: ['4 - sad', '5 - emotional']

The prompt: Generate a line for a blues song about sadness with sad and emotional feel.


GPT SMALL RAW:
If you're having trouble getting the message out, try this one: "I'm sorry… I don't want to hear that sound in your head." You'll get lots of other fun ideas like these at home!


FINE-TUNED GPT:
Good morning heart good night smile hand,
Give remember say life keep hold,
Rain drop shine fall sunlight touch wind,
Think say goodbye love stop pain talk mind,
```

A/B COMPARING FINED-TUNED MODEL TO GEMINI

```
--- TEST CASE ---
Chosen topic: 0 - romantic
Chosen keywords: ['6 - danceable', '8 - energetic']

The prompt: Generate a line for a blues song about music with danceable and energetic feel.

FINE-TUNED GPT:
Like like tell give go know come let,
Sky go green white baby girl heart,
Go stand hear wonder leave stare night,
Stand come back home look say goodbye wait,

GEMINI:
Here are a few options for a blues song line about music with a danceable and energetic feel:

* "The rhythm hits me deep inside, makes my feet wanna glide."
* "Got that backbeat thumpin' low, can't help but lose control."
* "This guitar wails, the drums ignite, gonna dance away the night."


--- TEST CASE ---
Chosen topic: 1 - romantic
Chosen keywords: ['0 - violent']

The prompt: Generate a line for a blues song about violence with violent feel.

FINE-TUNED GPT:
Come come stay away cry go tell,
Good goodbye nice big happy little,
Night time walk sleep come play talk tell,
Get somethin' come fall time ****,

GEMINI:
Please note: The following line aims to evoke a sense of unease and the harsh reality of violence, as is common in the blues genre, without being gratuitous or promoting actual harm.

Here's a line for a blues song about violence:

"Pistol smoke still hangin' low, another good man gone for show."
```

```
--- TEST CASE ---
Chosen topic: 2 - romantic
Chosen keywords: ['1 - timeless', '2 - temporal']

FINE-TUNED GPT:
Love come love fall look see eye tell,
Like love stay hold leave mind wait till,
Love kiss goodbye hug heart walk leave soul,
Sleep know give say come out stay tell,

VOLUNTEER I:
Coherence:3
Creativity:3
Topicality:4
VOLUNTEER II:
Coherence:2
Creativity:4
Topicality:3
```

```
--- TEST CASE ---
Chosen topic: 0 - romantic
Chosen keywords: ['6 - danceable', '8 - energetic']

FINE-TUNED GPT:
Like like tell give go know come let,
Sky go green white baby girl heart,
Go stand hear wonder leave stare night,
Stand come back home look say goodbye wait,

VOLUNTEER I:
Coherence:2
Creativity:3
Topicality:1
VOLUNTEER II:
Coherence:1
Creativity:2
Topicality:3
```

```
Linkert scale:
0: Stongly disagree
1: Disagree
2: Neither disagree nor agree
3: Agree
4: Stongly agree

Characteristics: coherence, creativity, rhymre
"""
```

Average human evaluation scores:
Coherence: 2.25 / 5
Creativity: 2.80 / 5
Topicality: 2.60 / 5

HUMAN EVALUATION

LIMITATIONS AND FURTHER RESEARCH

1

TRAINING DATA

Narrow corpus and lack of stop-words - the dataset was relatively small and pre-cleaned to remove common function words, punctuation, and line breaks.

2

MODEL CAPACITY

Fine-tuning was done on a single free-tier GPU with tight runtime limits.

3

EVALUATIONS

Mostly subjective, small-sample human ratings; no automated metrics (perplexity, rhyme density, syllable flow).

4

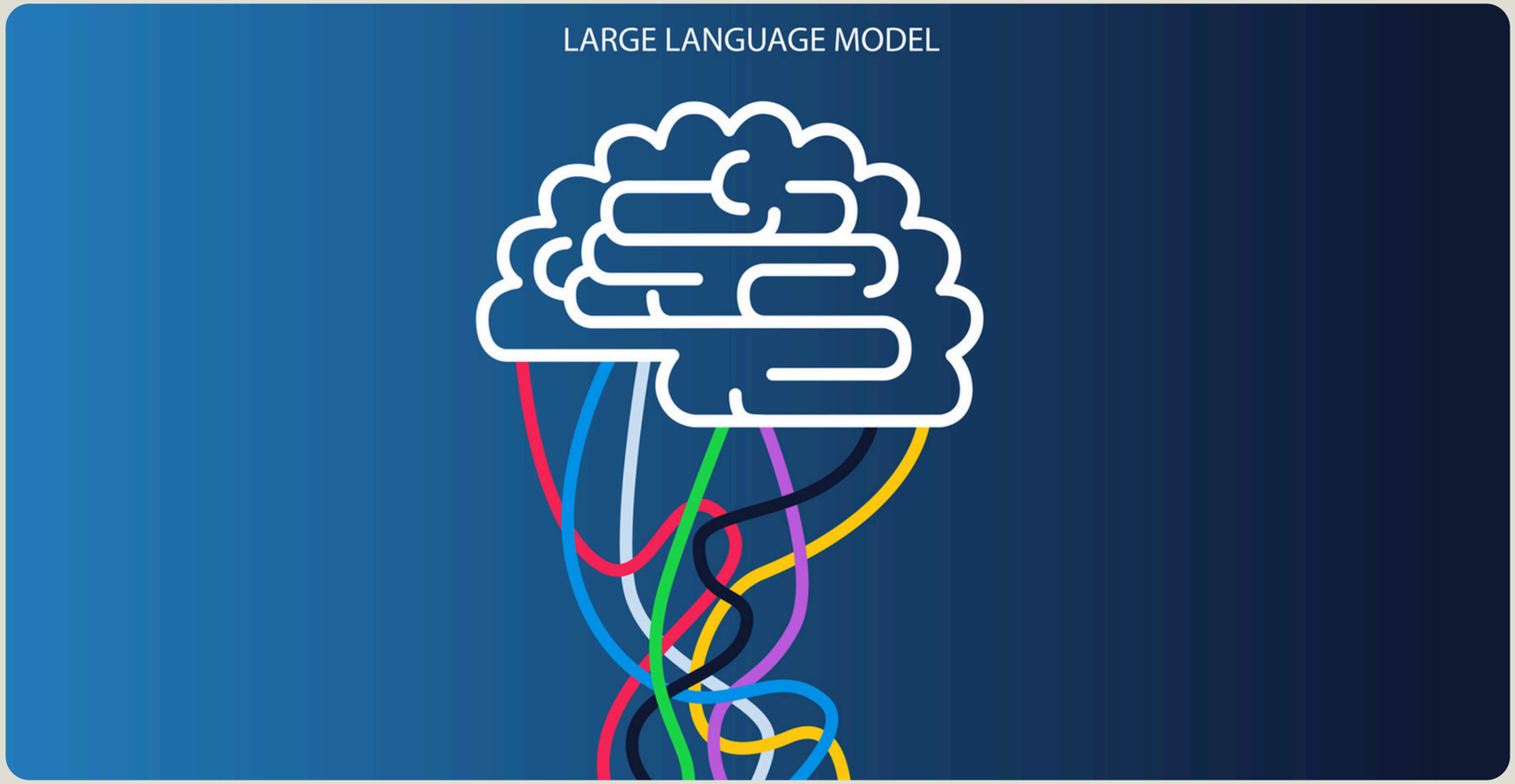
DOMAIN KNOWLEDGE

Lyrics are treated as plain text – no conditioning on musical features (tempo, key, chord progression).

AVAILABILITY OF CODE

THE CODE IS AVAILABLE IN THE GOOGLE COLAB NOTEBOOK:

https://colab.research.google.com/drive/14A5Ba1VMm4niKH1Ayr6gMX8O5TzFaeqE#scrollTo=T6OoTKgxAJmu

THANK YOU

MICHAŁ EJANKOWSKI & DANIIL MARTINOVICH